

The background of the slide features a large, faint watermark of the Universidad Mayor de San Sebastián (UMSS) logo. It is an inverted triangle with a red border. Inside, the top half is white with the letters 'UMSS' in blue. The bottom half is split into two red triangles meeting at a central point, where a small grey geometric logo is located.

UMSS

String Matching - Estructuras

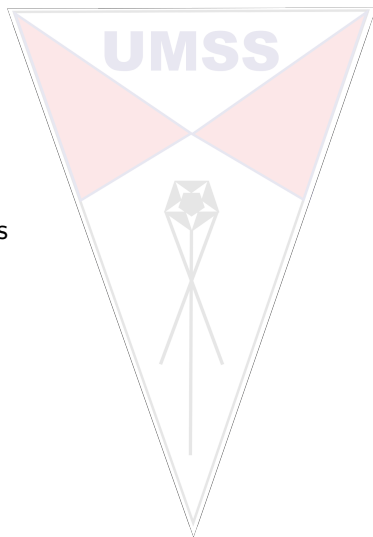
Leticia Blanco

Departamento de Informática - Sistemas
UMSS

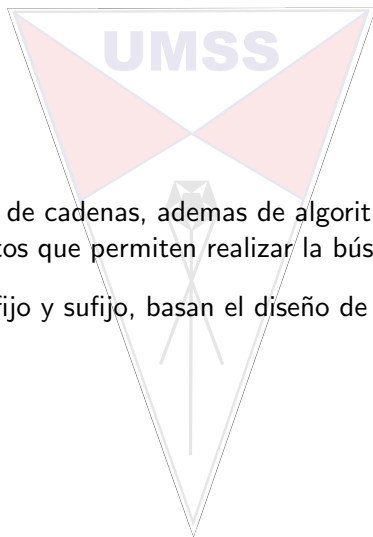
14 de noviembre de 2024

Contenido

- ▶ Generalidades
- ▶ Trie
- ▶ Suffix tree



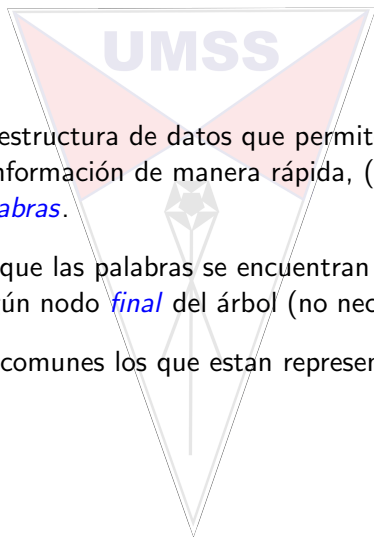
Generalidades



En el tratamiento de cadenas, además de algoritmos se tienen estructuras de datos que permiten realizar la búsqueda de palabras.

Conceptos de prefijo y sufijo, basan el diseño de un buen número de estructuras.

TRIE



Un TRIE, es una estructura de datos que permite realizar recuperación de información de manera rápida, (reTRIEval) en un *diccionario de palabras*.

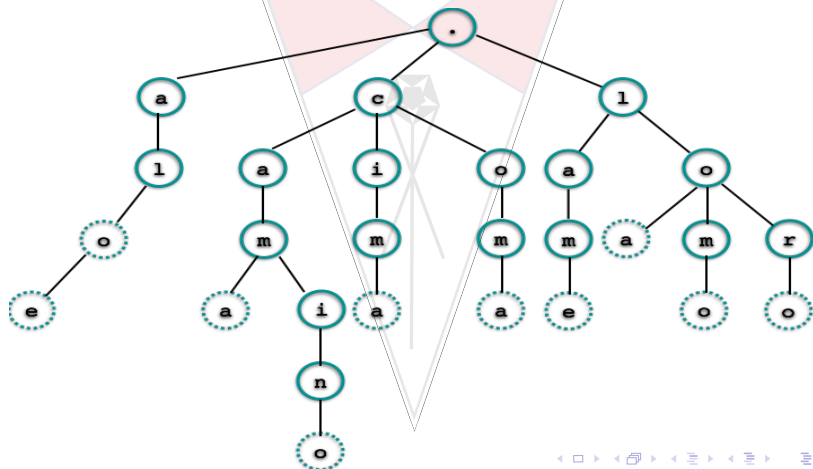
Es un árbol en el que las palabras se encuentran representadas desde la raíz a algún nodo *final* del árbol (no necesariamente hoja).

Identifica prefijos comunes los que estan representados en las ramas del arbol.

Presentan orden.

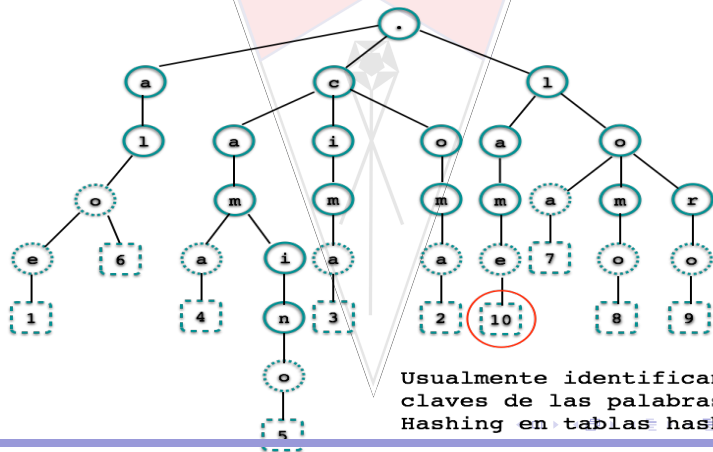
TRIE

aloe, coma, cima, cama, camino, alo, loa, lomo, loro, lame



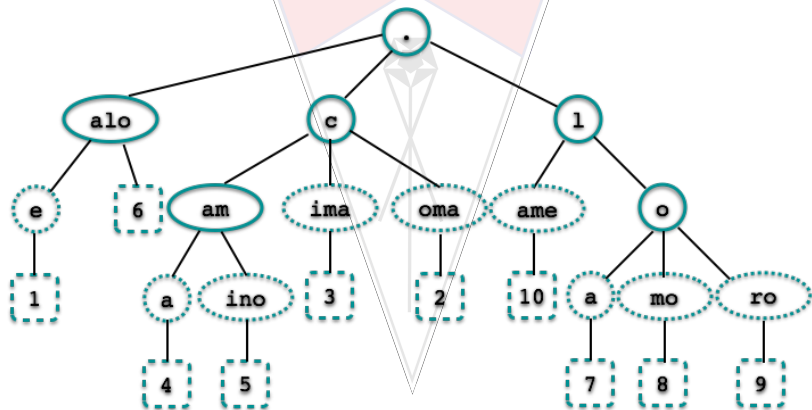
TRIE

aloe, coma, cima, cama, camino, alo, loa, lomo, loro, lame
1 2 3 4 5 6 7 8 9 10



TRIE - Compacto

aloe, coma, cima, cama, camino, alo, loa, lomo, loro, lame
1 2 3 4 5 6 7 8 9 10



TRIE - Análisis

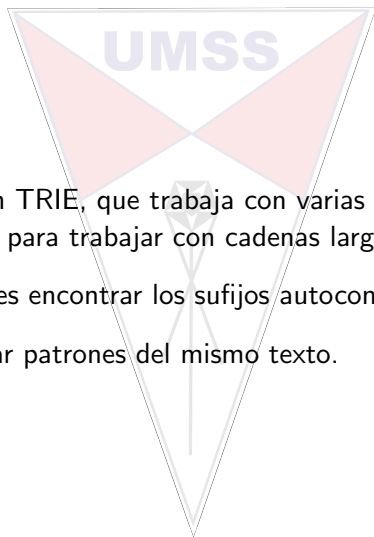
La construcción del árbol tiene complejidad $O(|\Sigma|^k n)$, donde Σ es el alfabeto, k es la cantidad de palabras y n es la longitud de la palabra mas larga.

Inserción de una nueva palabra en el TRIE tiene una complejidad $O(n)$, cuando n es la longitud de la palabra a añadir.

El buscar una palabra de tamaño m en el diccionario, tiene complejidad $O(m)$.

Inspecciona una rama a lo sumo.

Suffix Tree



A diferencia de un TRIE, que trabaja con varias cadenas cortas, un Suffix Tree es útil para trabajar con cadenas largas.

Y lo que se hace es encontrar los sufijos autocontenidos.

Útil para encontrar patrones del mismo texto.

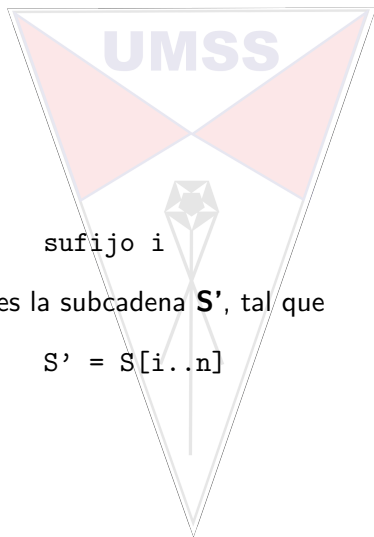
Generalidades

Se dice que el

sufijo i

de una cadena **S** es la subcadena **S'**, tal que

$$S' = S[i..n]$$



Generalidades

Si se tiene:

$S = \text{ABRACADABRA}$

$S' = \text{DABRA}$

S' es sufixo 7 de S , ya que
 $\text{DABRA} = S[7..11]$

Generalidades

El árbol tiene n hojas.

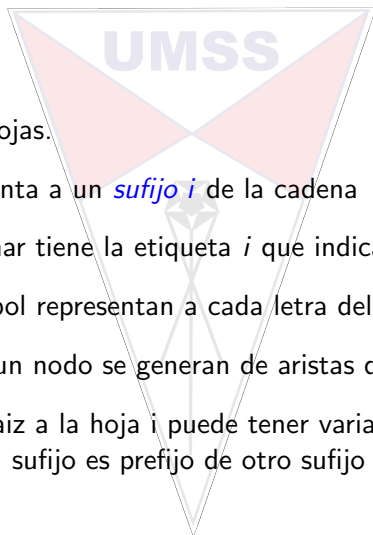
Cada hoja representa a un *sufijo i* de la cadena

La rama al terminar tiene la etiqueta i que indica el *sufijo i*

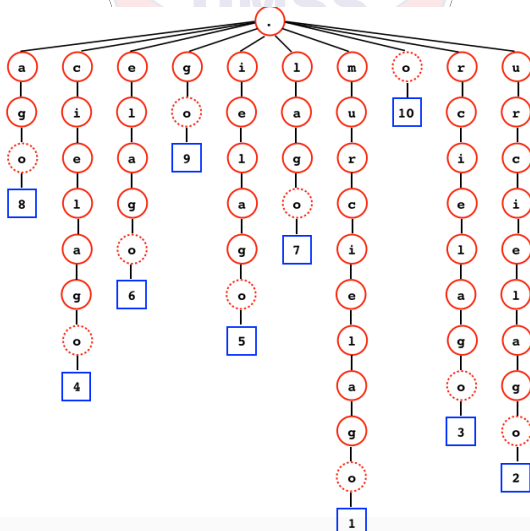
Las aristas del árbol representan a cada letra del alfabeto

Los sucesores de un nodo se generan de aristas de distintas letras

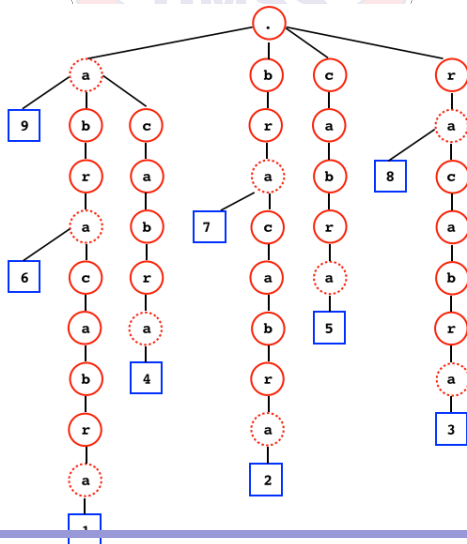
El camino de la raíz a la hoja i puede tener varias etiquetas, dependiendo si un sufijo es prefijo de otro sufijo



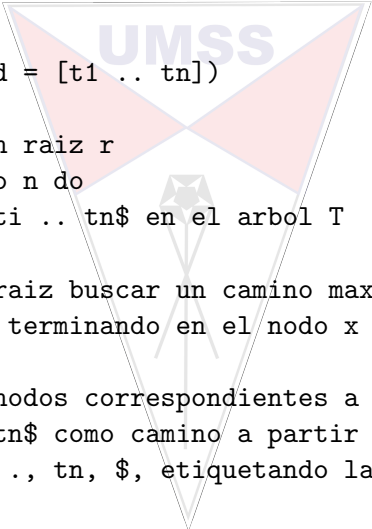
Suffix Tree - palabra murcielago



Suffix Tree - palabra abracabra



Suffix Tree



```
construir ( cad = [t1 .. tn])  
  t' = t$  
  T = arbol con raiz r  
  for i = 0 to n do  
    insertar ti .. tn$ en el arbol T
```

Desde la raiz buscar un camino maximo
ti. . . tj terminando en el nodo x

Ingresar nodos correspondientes a
tj+1. . . tn\$ como camino a partir de x con etiquetas
tj+1, . . . , tn, \$, etiquetando la hoja con i

```
return T (El arbol de sufijos de t1. . . tn$ )
```

Suffix Tree - análisis

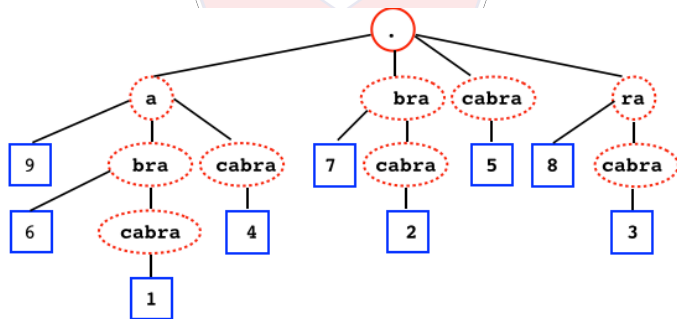
La construcción del árbol tiene complejidad $O(n^2 \log n)$

Construye niveles en el árbol como caracteres tenga la cadena.

Lo usual es que las aristas sean transiciones en un camino del árbol, estas transiciones son las ocurrencias de las letras

La búsqueda de un patrón en el texto, considerando que ya se tiene el árbol, tiene complejidad: $O(m \log n)$; donde m es el tamaño del patrón y n el tamaño del texto

Suffix Tree - compacto



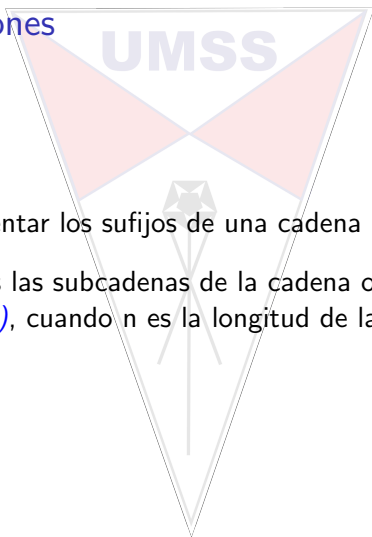
Suffix Tree - análisis

La construcción del árbol tiene complejidad aun de $O(n \log n)$ (Ukkonen)

El espacio que usa esta implementación es menor

La búsqueda tiene complejidad $m(|\Sigma| + \log n)$ La búsqueda de todas las ocurrencias del patron es $k \log n$; donde k es el numero de ocurrencias

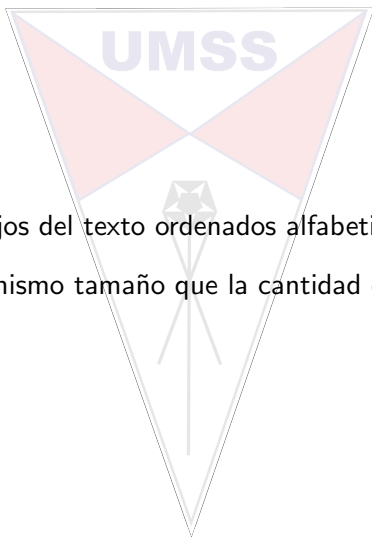
TRIE - Adaptaciones



TRIE para representar los sufijos de una cadena

Inserción de todas las subcadenas de la cadena original con una complejidad $O(n^2)$, cuando n es la longitud de la palabra a tratar.

Arreglo de sufijos



Se guarda los sufijos del texto ordenados alfabeticamente

El arreglo es del mismo tamaño que la cantidad de caracteres del texto

Arreglo de sufijos

Si se tiene:

`s` = ababaca

`sufs` = ababaca, babaca, abaca, baca, aca, ca, a

`i` = 1, 2, 3, 4, 5, 6, 7

Orden

`sufs` = a, ababaca, abaca, aca, babaca, baca, ca

`arrSufs` = 7, 1, 3, 5, 2, 4, 6

El arreglo es del mismo tamaño que la cantidad de caracteres del texto

Arreglo de sufijos

La búsqueda de un P en T , se realiza

sufs = a, ababaca, abaca, aca, babaca, baca, ca

arrSufs = 7, 1, 3, 5, 2, 4, 6

pos = 1, 2, 3, 4, 5, 6, 7

se quiere buscar aba

el primer sufijo que empieza con aba es ababaca
en la posicion 2

el ultimo sufijo que empieza con aba es abaca
en la posicion 3

Entonces hay dos ocurrencias: $3-2+1$

Arreglo de sufijos

La búsqueda de un P en T , se realiza

sufs = a, ababaca, abaca, aca, babaca, baca, ca

arrSufs = 7, 1, 3, 5, 2, 4, 6

pos = 1, 2, 3, 4, 5, 6, 7

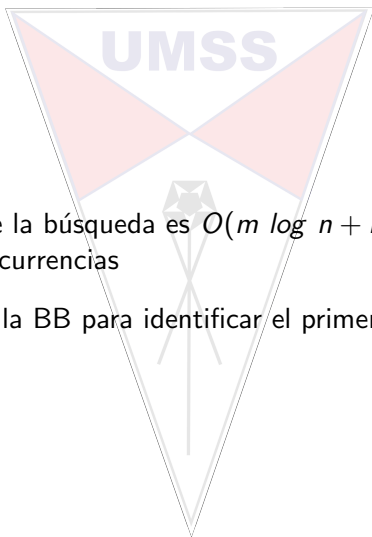
se quiere buscar a

el primer sufijo que empieza con a es a
en la posicion 1

el ultimo sufijo que empieza con a es aca
en la posicion 4

Entonces hay 4 ocurrencias: $4-1+1$

Arreglo de sufijos



La complejidad de la búsqueda es $O(m \log n + k \log n)$ donde k es el número de ocurrencias

Se puede adaptar la BB para identificar el primer y ultimo sufijo